CH01 - Time and Frequency Domain

1. Time Domain

The **time domain** represents how a signal changes over **time**.

A signal represented as a function of time

Time - `X Axis` : Amplitude - `Y Axis`

There are two types of time domain signals - ***Continuous time and Discrete time***

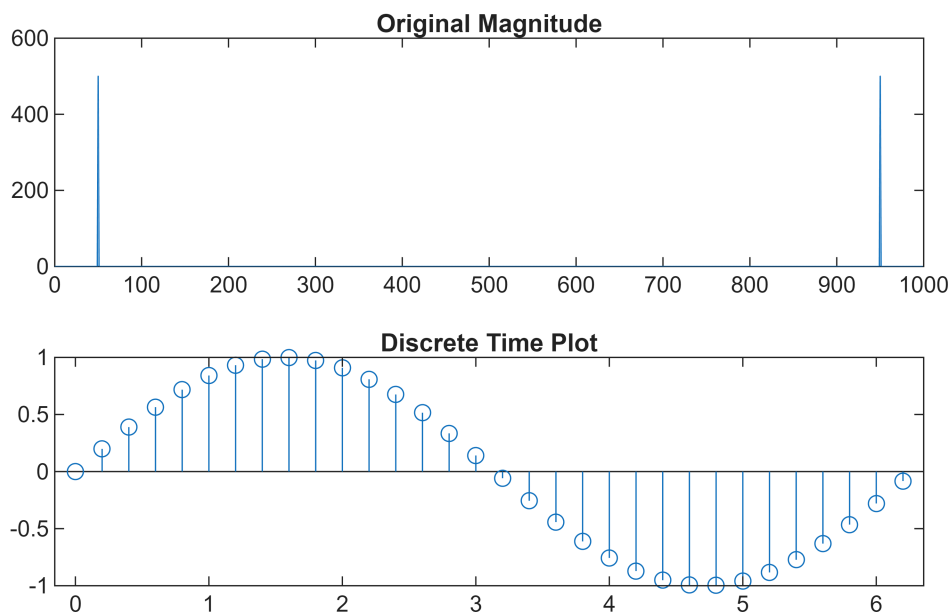
```
t = 0:0.2:2*3.14
```

```
t = 1×32
         0    0.2000    0.4000    0.6000    0.8000    1.0000    1.2000    1.4000 ···
```

```
y = sin(t)
```

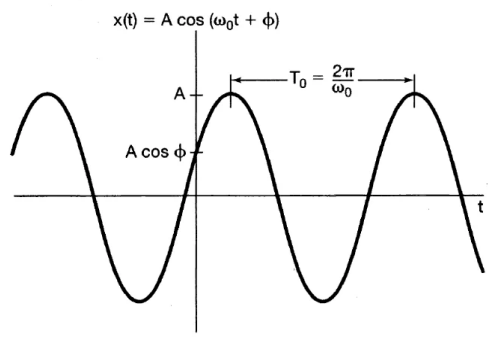
```
y = 1×32
         0    0.1987    0.3894    0.5646    0.7174    0.8415    0.9320    0.9854 ···
```

```
plot(t,y) %Continuous plot
title("Continuous Time Plot")
stem(t,y) %Discrete plot
title("Discrete Time Plot")
```



1.1 Continuous Time Signals (CT)

A continuous-time signal is defined for **every possible moment** in time. There are no gaps; between any two points in time, there are infinitely many more points.



Mathematical Notation: Written as ***x(t)*** , where `t` is a real number. It appears as a smooth, unbroken line or curve.

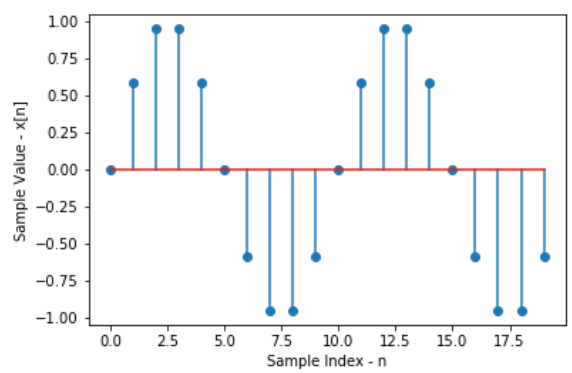
Real-World Example: Natural phenomena. The actual sound waves of your voice, the temperature throughout the day, or the voltage coming out of a battery.

Analogy: A **vinyl record**. The needle follows a continuous physical groove that represents the sound wave perfectly.

1.2 Discrete Time Signals (DT)

A discrete-time signal is defined only at **specific, isolated points** in time. It is essentially a sequence of numbers.

Mathematical Notation: Written as `x(n)` , where `n` is an integer (an index). It appears as a "lollipop" graph (called a **stem plot** in MATLAB), where vertical lines show the value at specific intervals.



Real-World Example: Digital technology. A digital audio file (MP3), your daily closing stock price, or heart rate readings taken every minute.

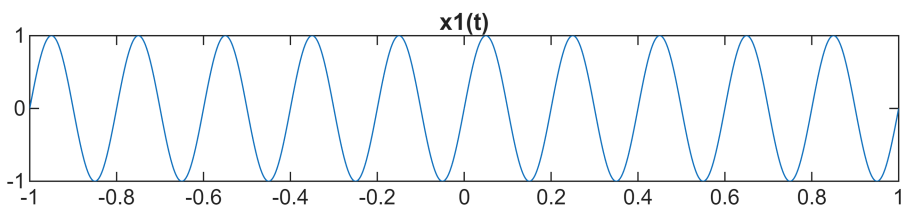
Analogy: A **digital photo**. If you zoom in enough, you see individual pixels (samples) rather than a smooth, continuous blur of color.

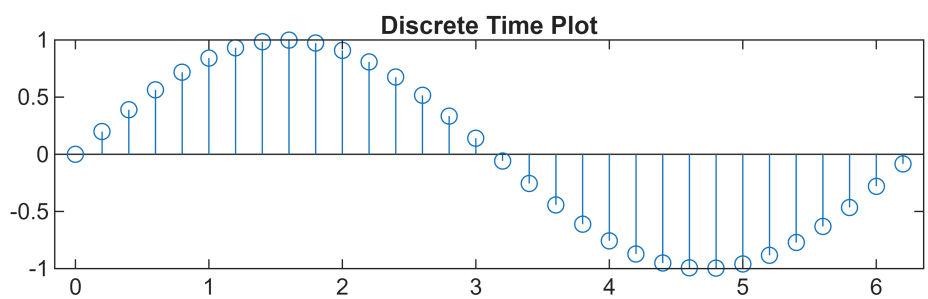
1.3 Properties of Time Domain Signals

1.3.1 Linearity

$$ax_1(t) + bx_2(t) \rightarrow ay_1(t) + by_2(t)$$

```
t = -1:0.001:1;

x1 = sin(2*pi*5*t);
x2 = cos(2*pi*3*t);

a = 2;
b = 0.5;

lhs = a*x1 + b*x2; % linear combination

subplot(3,1,1);
plot(t,x1); title('x1(t)');
```






```
subplot(3,1,2);
plot(t,x2); title('x2(t)');

subplot(3,1,3);
plot(t,lhs); title('a*x1 + b*x2');
```

1.3.2 Time Shifting

To shift the signal to right side (Delay the signal)

```
sin (2*pi*3*(t-t0))
```

To shift the signal to left side (Advance the signal)

```
sin (2*pi*3*(t+t0))
```

```
clc; clear; close all;

t = -1:0.001:1;

x = sin(2*pi*5*t);

t0 = 0.07; % NOT multiple of 0.2

x_shift1 = sin(2*pi*5*(t + t0));
x_shift2 = sin(2*pi*5*(t - t0));

subplot(3,1,1);
plot(t,x,'k','LineWidth',1.5);
title('Original x(t)');
grid on;
subplot(3,1,2);
plot(t,x_shift1,'r','LineWidth',1.5);
title('x(t + 0.07) → Left Shift');
grid on;
subplot(3,1,3);
plot(t,x_shift2,'b','LineWidth',1.5);
title('x(t - 0.07) → Right Shift');
grid on;
```

1.3.3 Time Scaling

To compress a signal

```
x(t) ------compression-----> x(n*t)
```

To Stretch a signal

```
x(t) ------compression-----> x(t/n)
```

```
clc; clear; close all;

t = -1:0.001:1;

x = sin(2*pi*5*t);

t0 = 0.07; % NOT multiple of 0.2

x_comp2 = sin(2*(2*pi*5*(t)));
```

```
x_comp4 = sin(4*(2*pi*5*(t)));
x_exp2 = sin((2*pi*5*(t))/2);
x_exp4 = sin((2*pi*5*(t))/4);

subplot(5,1,1);
plot(t,x_exp4,'r','LineWidth',1.5);
title('x(t/2) → Expansion');
grid on;

subplot(5,1,2);
plot(t,x_exp2,'b','LineWidth',1.5);
title('x(t/2) → Expansion');
grid on;

subplot(5,1,3);
plot(t,x,'k','LineWidth',1.5);
title('Original x(t)');
grid on;

subplot(5,1,4);
plot(t,x_comp2,'r','LineWidth',1.5);
title('x(2t) → Compression');
grid on;
subplot(5,1,5);
plot(t,x_comp4,'b','LineWidth',1.5);
title('x(4t) → Compression');
grid on;
```

1.3.4 Time Reversal

To reverse a signal

```
x(t) ------Reversal-----> x(-t)
```

```
clc; clear; close all;
t = -1:0.001:1;
x = sin(2*pi*5*t);

x_rev = sin(2*pi*5*(-t));

plot(t,x,'b','LineWidth',1.5); hold on;
plot(t,x_rev,'r--','LineWidth',1.5);

legend('Original','Reversed');
title('Time Reversal');
grid on;
```

1.3.5 Even and Odd Composition

In signal processing, every signal can be broken down into two simpler parts: an **Even** part and an **Odd part. This is like saying every number can be seen as a sum of other numbers—it's a way to simplify complex analysis.**

- **Even Signal (x_e):** Symmetric about the y-axis (a mirror image).
  - Formula: $x(t) = x(-t)$
  - Example: $\cos(t)$
- **Odd Signal (x_o):** Symmetric about the origin (rotated 180°).
  - Formula: $x(-t) = -x(t)$
  - Example: $\sin(t)$

Any signal $x(t)$ is the sum of its even and odd parts:

$$x(t) = x_e(t) + x_o(t)$$

To find these parts from any random signal, use these "averaging" formulas:

$$x_e(t) = \frac{x(t) + x(-t)}{2}$$

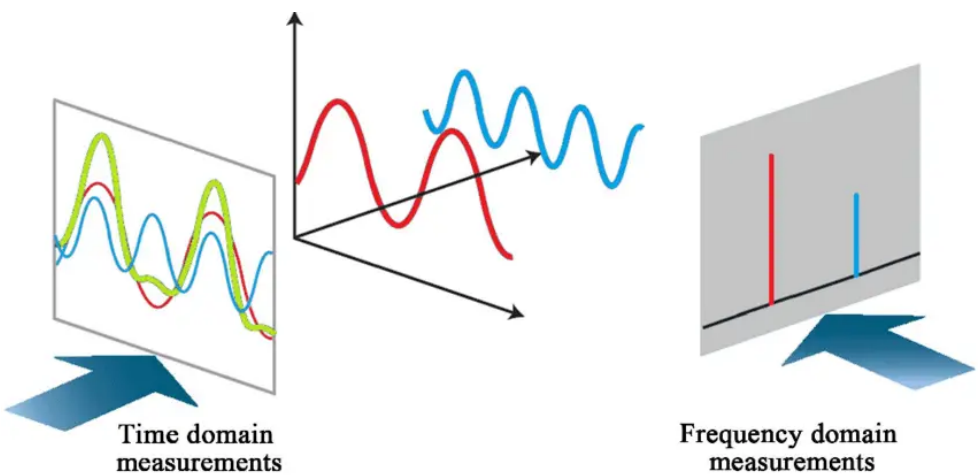
$$x_o(t) = \frac{x(t) - x(-t)}{2}$$

```
clc; clear; close all;
t = -1:0.001:1;

% 1. Define a signal that is NEITHER even nor odd
% Adding cos (even) and sin (odd) ensures both parts are non-zero.
x = cos(2*pi*5*t) + sin(2*pi*5*t) + sin (3*pi*2*t) + sin (7*pi*9*t);

% 2. Create the flipped version x(-t)
x_flipped = cos(2*pi*5*(-t)) + sin(2*pi*5*(-t)) + sin (3*pi*2*(-t))+ sin (7*pi*9*(-
t));

% 3. Extract components
xe = (x + x_flipped) / 2;
xo = (x - x_flipped) / 2;

% Plotting
figure('Name', 'Even and Odd Decomposition');

subplot(3,1,1);
plot(t, x, 'b', 'LineWidth', 1.5);
```

```
title('Original Signal x(t) = cos(10\pi t) + sin(10\pi t)');
grid on;

subplot(3,1,2);
plot(t, xe, 'g', 'LineWidth', 1.5);
title('Even Component xe(t) ');
grid on;

subplot(3,1,3);
plot(t, xo, 'm', 'LineWidth', 1.5);
title('Odd Component xo(t) ');
grid on;
```

2. Frequency Domain

The Frequency Domain is based on Fourier's Discovery: Any complex signal—no matter how messy it looks—is just a bunch of simple sine waves added together.



When you switch to the Frequency Domain, you are "un-mixing" the signal to see its ingredients:

- X-axis: Frequency (measured in Hertz, Hz).
- Y-axis: Amplitude (how strong that specific frequency is).

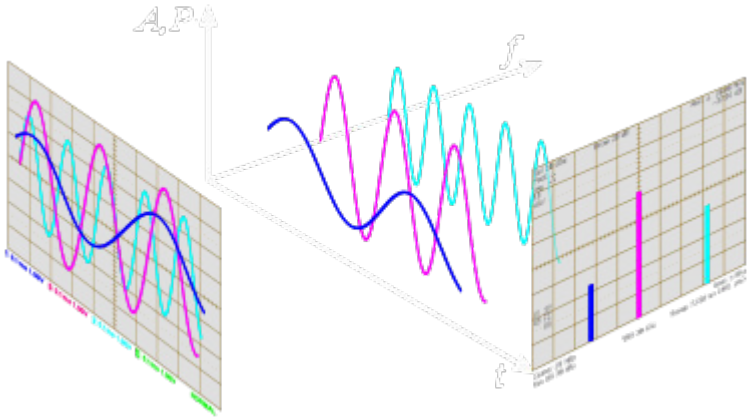
2.1 Why F-Domain?

Some problems are impossible to solve in the Time Domain but are incredibly easy in the Frequency Domain:

Noise Removal: If you have a recording with a high-pitched hiss, it's hard to "rub out" in the time domain. In the frequency domain, that hiss shows up as a separate spike at a high frequency. You can just "delete" that spike.

Compression: MP3s work by looking at the frequency domain and deleting the frequencies that human ears can't hear.

System Analysis: It tells you if your system (like a speaker) prefers "Bass" (low frequencies) or "Treble" (high frequencies).



2.2 Frequency plot of a music file

```
clc; clear; close all;

[x, fs] = audioread("E:\signalprocessing\Hindolam.wav"); % put your file here

% If stereo → convert to mono
if size(x,2) > 1
    x = mean(x,2);
end

t = (0:length(x)-1)/fs;

% sound(x, fs); % play original

figure;
plot(t, x);
title('Time Domain Signal');
xlabel('Time (s)');

N = length(x);

X = fft(x);
f = (0:N-1)*(fs/N);

figure;
plot(f, abs(X));
title('Frequency Spectrum');
xlabel('Frequency (Hz)');
xlim([0 fs/2]); % important
```

Now let's apply a low pass filter to this song and test its output

```
clc; clear; close all;
```

```
[x, fs] = audioread("E:\signalprocessing\Hindolam.wav");

% Convert to mono if stereo
if size(x,2) > 1
    x = mean(x,2);
end

t = (0:length(x)-1)/fs;
N = length(x);

% FFT
X = fft(x);

% Frequency axis
f = (0:N-1)*(fs/N);

% -------------------------------
% 🔹 LOW PASS FILTER
% -------------------------------
cutoff = 500; % Hz

H = zeros(N,1);

% Proper symmetric filtering
H(f < cutoff) = 1;
H(f > fs - cutoff) = 1;

X_lp = X .* H;

x_lp = real(ifft(X_lp));

% Normalize (important for sound)
x_lp = x_lp / max(abs(x_lp));

sound(x_lp, fs);

% -------------------------------
% 🔹 TIME DOMAIN PLOT
% -------------------------------
figure;

subplot(2,1,1);
plot(t, x);
title('Original Signal (Time)');
xlabel('Time (s)');

subplot(2,1,2);
plot(t, x_lp);
title('Low Pass Output (Time)');
xlabel('Time (s)');
```

```
% -------------------------------
% ⬇ FREQUENCY DOMAIN PLOT
% -------------------------------
f_plot = f(1:N/2);

X_mag = abs(X(1:N/2));
X_lp_mag = abs(X_lp(1:N/2));

figure;

subplot(2,1,1);
plot(f_plot, X_mag);
title('Original Spectrum');
xlabel('Frequency (Hz)');
ylabel('|X(f)|');
xlim([0 5000]);
grid on;

subplot(2,1,2);
plot(f_plot, X_lp_mag);
title('Filtered Spectrum (Low Pass)');
xlabel('Frequency (Hz)');
ylabel('|X(f)|');
xlim([0 5000]);
grid on;
```

2.3 Properties of F Domain

2.3.1 Linearity

$$ax_1(t) + bx_2(t) \leftrightarrow aX_1(f) + bX_2(f)$$

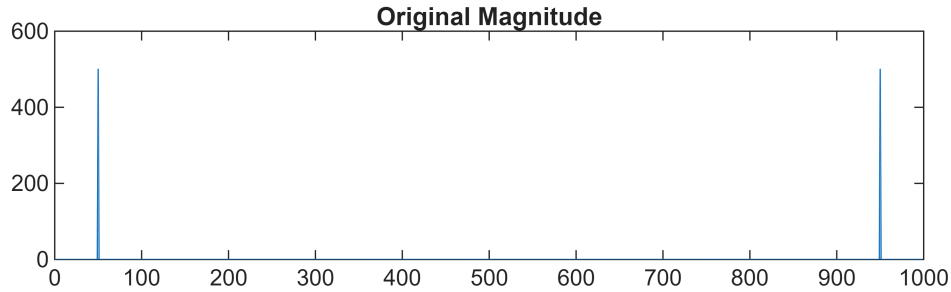
```
clc; clear; close all;

fs = 1000;
t = 0:1/fs:1-1/fs;

x1 = sin(2*pi*50*t);
x2 = cos(2*pi*120*t);

N = length(t);
f = (0:N-1)*(fs/N);

a = 2;
b = 0.5;
```

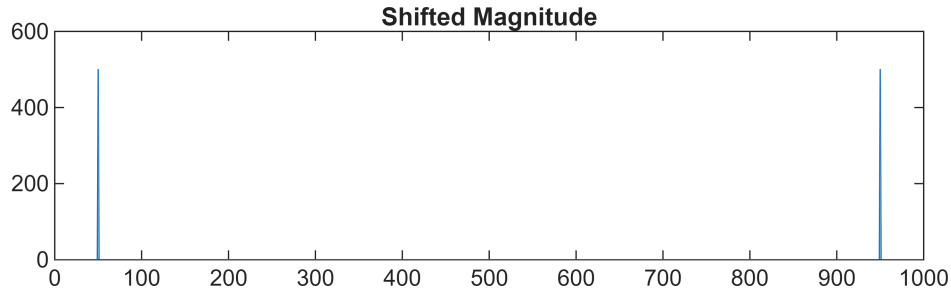
```
x = a*x1 + b*x2;

X1 = fft(x1);
X2 = fft(x2);
X  = fft(x);

lhs = X;
rhs = a*X1 + b*X2;

subplot(2,1,1);
plot(f, abs(lhs)); title('FFT of a*x1 + b*x2');

subplot(2,1,2);
plot(f, abs(rhs)); title('a*X1 + b*X2');

%xlim([0 200]);
```

2.3.1 Time Shifting

$$x(t - t_0) \leftrightarrow X(f)e^{-j2\pi f t_0}$$

```
t0 = 0.01;

x_shift = sin(2*pi*50*(t - t0));

X  = fft(x1);
Xs = fft(x_shift);

subplot(2,1,1);
plot(f, abs(X)); title('Original Magnitude');

subplot(2,1,2);
plot(f, abs(Xs)); title('Shifted Magnitude');
```



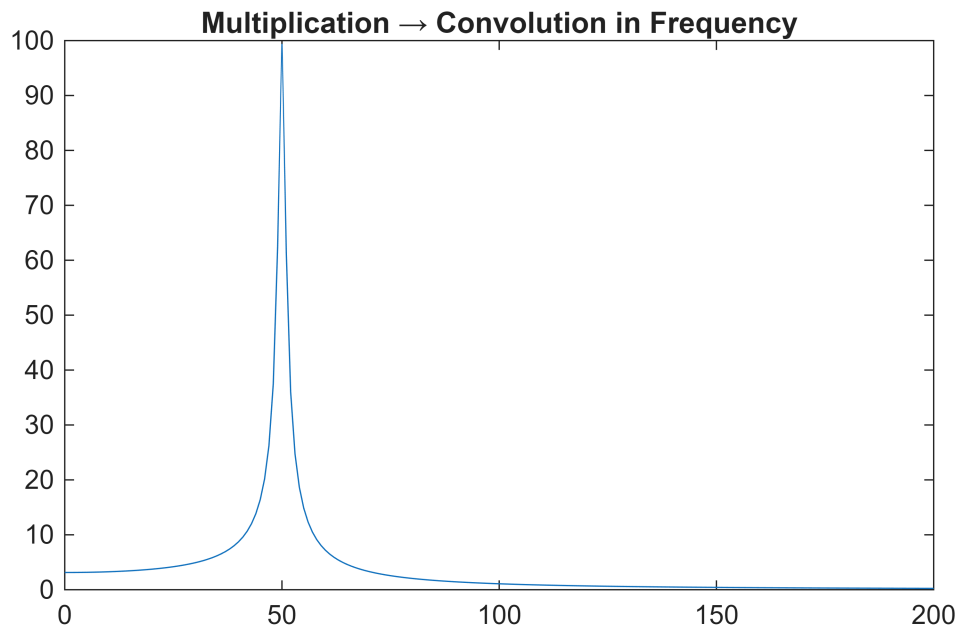



```
%xlim([0 200]);
```

2.3.3 Multiplication Property

$$x(t)h(t) \leftrightarrow X(f) * H(f)$$

```
h = exp(-5*t);
y = x1 .* h;

Y = fft(y);

figure;
plot(f, abs(Y));
title('Multiplication → Convolution in Frequency');
xlim([0 200]);
```


Multiplication → Convolution in Frequency
100
90
80
70
60
50
40
30
20
10
0
0
50
100
150
200